

## Phase 1: Obtain Google API Credentials

Before writing code, you must register your web application on the [Google Cloud platform](#).

1. Go to the Google Cloud Console.
  2. **Create a new project** or select an existing one from the top left.
    - For a **new project** type any **project name** and select **no organisation** in **location** label then click **create** button.
  3. Navigate to **APIs & Services > OAuth consent screen > click get started**
    - Fill out the mandatory fields: **App name**, **User support email**, and **Developer contact information**.
    - Select **External** (if your app is for public use) and click **Create**.
    - Fill out the **contact information**.
    - Click **create**.
  4. Navigate to **APIs & Services > Credentials**:
    - Click **Create Credentials** and choose **OAuth client ID**.
    - Set the Application Type to **Web application**.
    - Under **Authorized JavaScript origins**, add your local server URL: `http://localhost:3000`.
    - Under **Authorized redirect URIs**, add your application's expected callback path: `http://localhost:3500/googlesigncallback`
    - Click **Create** to receive your **Client ID** and **Client Secret**.
  5. Go to **audience section** there will be **publishing status** with options
    - **Testing** (Allows Only users added to your "Test users" list.)
    - **Publish** (Allows Any users with a valid Google account.)
  6. **Configure the "Test users" Section**
    - Because your app is in **Testing** status, You must explicitly whitelist your own accounts.
    - Click the **+ ADD USERS** button and enter your personal Gmail address, plus any other developer or QA email addresses.
    - Limit:100 test users during the testing phase.
-

## What Happens If You Don't Add Test Users?

If you attempt to click "**Login with Google**" in your Express app using an unlisted email account, Google will block the request and display a "**Sign in with Google temporary disabled for this app**" error page with **Error 403: access\_denied**.

**passport.initialize()** is a mandatory middleware function that prepares Passport.js to handle incoming HTTP requests and manage user authentication.

Here is exactly what it does behind the scenes:

### 1. Initializes the Passport Framework

It attaches the Passport instance to the incoming Express request (req) object. This injects standard Passport helper functions directly into your route handlers, allowing you to call methods like:

- req.login()
- req.logout()
- req.isAuthenticated() [[1](#), [2](#)]

### 2. Creates an Authentication Context

It sets up an isolated, internal space within the request lifecycle to store authentication state, strategies, and configuration details for that specific request. Without this initialization step, Express cannot understand or execute passport.authenticate()

strategies.

### 3. Works Alongside Sessions

While passport.initialize() sets up the core framework, it works as a team with passport.session(). [[1](#), [2](#)]

- **passport.initialize()**: Bootstraps the framework on every single request.
- **passport.session()**: Alters the request context *only* if your app uses persistent login sessions, checking the session cookie to alter req.user automatically. [[1](#), [2](#)]

If you are building an API that uses **JSON Web Tokens (JWT)** instead of cookies/sessions, you still must include passport.initialize(), but you can safely omit passport.session(). [[1](#)]

Would you like to see how to configure Passport for **stateless JWT authentication** instead of session cookies, or do you want to troubleshoot a specific **error** you are seeing with your current setup?

Passport is **not strictly required** to use Google OAuth, but developers choose it because it replaces a massive amount of manual, complex network logic with a few clean lines of code.

Here is exactly what Passport does for you that you would otherwise have to build, test, and secure yourself.

### 1. It Automates the 3-Step OAuth Handshake

Without Passport, you have to manually code the entire multi-step exchange required by Google's API:

- **The Redirect:** Building a complex authorization URL with the correct scopes, redirect URIs, and security states.
- **The Authorization Code:** Setting up an endpoint to receive Google's temporary authorization code.
- **The Token Exchange:** Writing manual fetch or axios POST requests to Google's token endpoint, sending your Client Secret securely to exchange that code for an access\_token. [[1](#), [2](#)]

Passport handles this entire handshake internally behind a single passport.authenticate('google') command.

### 2. It Normalizes the User Profile

Every OAuth provider (Google, GitHub, Facebook) formats user profile data (names, IDs, emails) differently in their raw API responses.

- Passport's Google Strategy automatically parses Google's specific, messy JSON response.
- It converts it into a clean, predictable, standard **User Profile Object** (profile.id, profile.displayName, profile.emails).
- This allows you to swap or add other login providers later without changing your database logic.

### 3. It Integrates Directly into Express Middleware

Passport blends perfectly into the Express lifecycle. Instead of writing custom logic on every route to see if a user is logged in, Passport instantly gives you:

- **req.isAuthenticated():** A simple true/false check to guard routes.
- **req.user:** Global access to the logged-in user's profile across any route file.